

TubeForge

Install & Usage Guide
Download -> Convert -> Play Anywhere.

1. What TubeForge is

TubeForge is a Flask + yt-dlp utility designed for self-hosted use on a local network. It accepts a single YouTube video URL, fetches metadata and available qualities, then downloads MP4 video or MP3 audio in a background job.

The interface is intentionally account-free and ad-free. The intended workflow is: Paste URL → Check Video → Choose format/quality → Download.

2. Architecture at a glance

```
Browser → Flask routes → jobs.py background job store → downloader.py → yt-dlp / FFmpeg  
→ temporary file → browser download
```

The Flask routes remain thin: they validate input, delegate download work, translate downloader errors into JSON, and return results. The heavy media work lives in downloader.py; validation and sanitization live in utils.py.

3. Local Windows installation

Recommended baseline: Python 3.11, FFmpeg, and a modern browser.

Create the virtual environment:

```
python -m venv venv  
.\venv\Scripts\Activate.ps1
```

Install Python dependencies:

```
python -m pip install --upgrade pip  
pip install -r requirements.txt
```

Verify FFmpeg:

```
ffmpeg -version
```

Copy .env.example to .env and adjust settings if needed. The default host is 0.0.0.0 and the default port is 5000.

Start the application:

```
python app.py
```

Then open <http://127.0.0.1:5000>. Other devices on the same Wi-Fi can use <http://YOUR-PC-IP:5000>.

4. Docker installation

The supplied Dockerfile uses Python 3.11-slim and installs FFmpeg and aria2. The Compose file maps host port 5000 to container port 5000 and is configured to restart unless stopped.

Build and start:

```
docker compose up -d --build
```

Check status and logs:

```
docker compose ps
docker compose logs -f
```

Stop:

```
docker compose down
```

5. Configuration

Variable	Default	Purpose
HOST	0.0.0.0	LAN-accessible bind address
PORT	5000	Flask HTTP port
DEBUG	False	Development debug mode
MAX_DURATION_SECONDS	blank	Optional duration cap
COOKIES_FROM_BROWSER	blank	Optional browser cookie source
COOKIES_FILE_PATH	blank	Optional Netscape cookie file
USE_ARIA2C	False	Enable aria2 external downloader

6. Using the application

Step 01 — Paste URL

Paste a single YouTube video URL into the TubeForge interface.

Step 02 — Check Video

TubeForge sends the URL to POST /formats. This stage is metadata-only: it does not download the media. The response supplies the title, thumbnail, and distinct available video heights.

Step 03 — Choose format and quality

Select MP4 Video or MP3 Audio. For MP4, a requested quality acts as an upper height limit; the source does not have to contain an exact matching stream.

Step 04 — Download

The browser creates a background job through POST /jobs, then polls GET /jobs//progress. When status becomes done, it fetches GET /jobs//result.

7. Progress behavior

MP4 downloads can contain separate video and audio streams. TubeForge tracks downloaded bytes for each stream and calculates one aggregate percentage. This prevents a visible reset when yt-dlp switches from the video stream to the audio stream.

Example: video 1.97 MiB / 1.97 MiB plus audio 2.16 MiB / 4.33 MiB gives 4.13 MiB downloaded out of 6.30 MiB total, or approximately 65.6%.

8. Cookies and YouTube sign-in errors

Cookies are optional and should only be used when YouTube starts returning sign-in or suspected-bot errors. The project supports a browser cookie source and a static Netscape-format cookie file; the static file takes priority when both are configured.

Example:

```
COOKIES_FROM_BROWSER=firefox
```

or:

```
COOKIES_FILE_PATH=C:\path\to\cookies.txt
```

Security: cookie files can contain active session credentials. Keep them private, never commit them to Git, and do not include them in Docker build context. The supplied project .gitignore already excludes cookies.txt.

9. Troubleshooting

- **Cannot open from another device:** confirm HOST=0.0.0.0, use the PC's LAN IP, and check Windows Firewall/private network permissions.
- **FFmpeg missing:** run `ffmpeg -version` and install/add FFmpeg to PATH. Docker installs FFmpeg automatically.
- **YouTube asks you to sign in:** update yt-dlp and, only if necessary, configure cookies.
- **Progress appears to restart:** make sure downloader.py, jobs.py, app.py, and the frontend JavaScript are from the same project version; the current backend uses aggregate stream progress.
- **Docker is showing old code:** rebuild with `docker compose up -d --build` and inspect `docker compose logs -f`.
- **Download reaches the end but does not finish:** the post-download stage may still be merging or converting with FFmpeg; inspect server/container logs.

10. API reference

Method	Endpoint	Purpose
GET	/	Main interface
GET	/health	Health check
POST	/formats	Metadata + qualities
POST	/jobs	Create background job
GET	/jobs/<id>/progress	Live job state
GET	/jobs/<id>/result	Fetch completed file

11. Development notes

Job state is held in a thread-safe in-memory JobStore. This is intentionally simple for a single-process LAN deployment. A multi-worker deployment would need shared job state such as Redis.

Each download gets its own UUID-named temporary directory. On successful result delivery, the file is read, the temporary directory is cleaned, and the job is removed from the store.

The application uses an allowlist of YouTube domains and sanitizes generated filenames. The downloader calls yt-dlp through its Python API rather than passing user input to a shell command.

12. Testing

Install development requirements:

```
pip install -r requirements-dev.txt
pytest
```

The current development dependency file includes pytest 8.3.2. Manual checks are documented in tests/MANUAL_CHECKLIST.md.

13. Responsible use

Use TubeForge only for media you are authorized to download and in accordance with applicable law, copyright rules, and YouTube's terms. The application does not grant rights to download or redistribute protected material.

Documentation source note

This guide is based on the supplied TubeForge project files, including app.py, downloader.py, jobs.py, config.py, utils.py, requirements files, Dockerfile, docker-compose.yml, .env.example, and .gitignore.